Project Title : Exploratory Data Analysis of Dubai's Real Estate Development Projects

---

Project Overview :

This project focuses on performing an **end-to-end Exploratory Data Analysis (EDA)** on real estate development projects in Dubai to uncover meaningful insights about **construction trends, project completion, and development patterns**.

The dataset consists of **3000+ records** with multiple features such as **project status, construction status, developer details, project type, number of units, buildings, villas, and completion percentage**. The analysis aims to transform raw data into **actionable insights** through systematic **data cleaning, preprocessing, and visualization**.

Relevance of the Study :

The real estate sector plays a significant role in the economic growth and urban development of Dubai. Analyzing real estate project data helps in understanding **market trends, development patterns, and project performance**. These insights are valuable for **investors, developers, and policymakers** to make informed decisions and improve planning strategies.This topic was chosen to work with **real-world data** and apply key data analytics concepts such as **data cleaning, preprocessing, exploratory data analysis, and visualization**. It also provides an opportunity to gain insights into Dubai’s rapidly growing real estate sector.

Data Source :

The dataset used in this project is obtained from the **Dubai Open Data Portal**:🔗 https://data.dubai/en/l/467654
The dataset contains detailed information about real estate development projects, including project status, type, location, and completion details, making it suitable for comprehensive data analysis.

Methodology and Data Analysis Process :

The project begins with **data loading and initial exploration using Pandas** to

understand the structure, data types, and key characteristics of the dataset.

This is followed by extensive **data preprocessing**, including:

- Handling missing values
- Removing duplicates
- Correcting data types
- Creating new derived features such as **project duration** and **project start year**

Further, **exploratory data analysis (EDA)** is performed to identify patterns, trends, and relationships between variables. This includes:

- Univariate analysis
- Bivariate analysis
- Multivariate analysis
- Grouping, aggregation, and correlation techniques

To enhance understanding, various **visualizations** are created using **Matplotlib,Seaborn and Plotly** and insights can support **better decision-making** for stakeholders in the real estate sector.

In [190…

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Loading the dataset using Pandas and displaying the first 10 rows to underst
df = pd.read_csv(r"C:\Users\hp\Desktop\DATA ANALYTICS\MAIN PROJECT\Exploratory
df.head(10)
```

Out[190…

	Zoning Authority	Project Status	Developer Name	Project Type	Project Name	Project End Date	I
0	Dubai Development Authority	FINISHED	TECOM Investments Free Zone LLC	buildings	Celia Residence	14-09-2024	E NB
1	Licenses	ACTIVE	Nakheel Company (P.S.C.)	villas	Jebel Ali Townhouses Village - Phase 3	31-12-2025	M (Pub Co
2	Dubai Development Authority	FINISHED	Front Line Investment Management LLC	villas	Damac Hills (2) - Baswood	21-03-2022	Ab Com
3	Dubai Development Authority	ACTIVE	Emaar Properties (P.S.C.)	buildings	Benghatti Skyblade	30-11-2027	(Pub Co
4	Licenses	FINISHED	Jumeirah Village (LLC)	buildings	Cappadocia	15-08-2013	F Dha
5	Dubai Development Authority	ACTIVE	Dubai Sports City (LLC)	buildings	Sports from Danube	30-03-2026	Arab
6	Licenses	ACTIVE	Jumeirah Village (LLC)	buildings	The Vine Residences	25-11-2026	Ab Com
7	Licenses	NOT_STARTED	Al Farjan (LLC)	villas	Tory by IGO	30-04-2027	F Dha
8	Dubai Development Authority	FINISHED	Dubai Properties (LLC)	buildings	Sundus Rosa	31-12-2010	(Pub Co
9	Licenses	PENDING	Nakheel Company (P.S.C.)	buildings	No crystal	30-09-2028	

In [191…

```
# Displays the number of columns and rows in the dataset
df.shape
```

Out[191… (3039, 15)

This shows that the dataset has `3039` rows and `15` columns.

In [192…
```
# Displays the data types of each column in the dataset.
df.dtypes
```

Out[192…
```
Zoning Authority       object
Project Status         object
Developer Name         object
Project Type           object
Project Name           object
Project End Date       object
Escrow Agent Name      object
No:of buildings         int64
Project Status.1       object
Project Start Date     object
Master Project         object
No:of Units             int64
No:of Villas            int64
Area Name              object
Percent Completed     float64
dtype: object
```

This shows the datatypes of each column in the dataset . In this project start date and end date datatype should be changed to `datetime`.

In [193…
```
# Provides a summary of the dataset including column names, non-null counts, a
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3039 entries, 0 to 3038
Data columns (total 15 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Zoning Authority    3039 non-null   object
 1   Project Status      3039 non-null   object
 2   Developer Name      3039 non-null   object
 3   Project Type        3039 non-null   object
 4   Project Name        3039 non-null   object
 5   Project End Date    2763 non-null   object
 6   Escrow Agent Name   3039 non-null   object
 7   No:of buildings     3039 non-null   int64
 8   Project Status.1    3039 non-null   object
 9   Project Start Date  3034 non-null   object
 10  Master Project      2749 non-null   object
 11  No:of Units         3039 non-null   int64
 12  No:of Villas        3039 non-null   int64
 13  Area Name           3039 non-null   object
 14  Percent Completed   508 non-null    float64
dtypes: float64(1), int64(3), object(11)
memory usage: 356.3+ KB
```

The dataset contains missing values that require preprocessing.

In [194…
```
# Generates descriptive statistics such as mean, minimum, maximum, and standar
df.describe()
```

Out[194…

	No:of buildings	No:of Units	No:of Villas	Percent Completed
count	3039.000000	3039.000000	3039.000000	508.000000
mean	1.372491	232.064166	24.174070	58.562992
std	4.488752	369.551083	106.475561	44.608819
min	0.000000	0.000000	0.000000	0.000000
25%	1.000000	9.000000	0.000000	3.750000
50%	1.000000	141.000000	0.000000	88.000000
75%	1.000000	300.000000	0.000000	100.000000
max	199.000000	10853.000000	1787.000000	100.000000

In [195…
```
# Identifies the number of missing values in each column.
df.isnull().sum()
```

Out[195…
```
Zoning Authority         0
Project Status           0
Developer Name           0
Project Type             0
Project Name             0
Project End Date       276
Escrow Agent Name        0
No:of buildings          0
Project Status.1         0
Project Start Date       5
Master Project         290
No:of Units              0
No:of Villas             0
Area Name                0
Percent Completed     2531
dtype: int64
```

In [196…
```
# Checks for duplicate rows in the dataset.
df.duplicated().sum()
```

Out[196…
```
np.int64(0)
```

`No duplicate rows` were found in the dataset

In [197…
```
# Replace all VALUE errors
df.replace(["#VALUE", "#VALUE!"], np.nan, inplace=True)

# Fill categorical nulls
```

```
df["Zoning Authority"].fillna("Unknown", inplace=True)

# Fix project status spelling
df["Project Status"] = df["Project Status"].replace({"FRIEZED": "FREEZED"})
```

Handling Invalid Values and Standardizing Categorical Data

- **Replaced Invalid Placeholder Values**
  - Identified entries such as `#VALUE` and `#VALUE!` in the dataset
  - Converted them to **NaN** to enable proper handling of missing data
- **Handled Missing Values in Categorical Columns**
  - Filled missing values in the `zoning_authority` column with **"Unknown"**
  - Ensures consistency and prevents errors during grouping and analysis
- **Corrected Inconsistent Category Labels**
  - Fixed misspelled values in the `project_status` column
  - Example: `FRIEZED` → `FREEZED`
  - Helps avoid duplicate categories and improves analysis accuracy

In [198…

```
# Converting the Project Start Date and Project End Date columns into datetime
df["Project Start Date"] = pd.to_datetime(df["Project Start Date"], errors="co
df["Project End Date"] = pd.to_datetime(df["Project End Date"], errors="coerce
df.dtypes
```

Out[198…

```
Zoning Authority              object
Project Status                object
Developer Name                object
Project Type                  object
Project Name                  object
Project End Date      datetime64[ns]
Escrow Agent Name             object
No:of buildings                int64
Project Status.1              object
Project Start Date    datetime64[ns]
Master Project                object
No:of Units                    int64
No:of Villas                   int64
Area Name                     object
Percent Completed            float64
dtype: object
```

The project start and end date is converted to datetime for better analysis.

In [199…

```
# Drop rows with missing start date
df.dropna(subset=["Project Start Date"], inplace=True)

# Fill Master Project
df["Master Project"].fillna("Unknown", inplace=True)

# Fill Percent Completed
df["Percent Completed"].fillna(0, inplace=True)

# Fill missing Project End Date with today's date
df["Project End Date"].fillna(pd.Timestamp.today(), inplace=True)

# Create Project Duration
df["Project Duration (Days)"] = (df["Project End Date"] - df["Project Start Da
```

Data Cleaning and Feature Engineering Steps

- **Remove missing start dates**
  - Removes rows where `*Project Start Date*` is missing
  - Ensures accurate calculation of project duration
- **Fill missing Master Project values**
  - Replaces null values with `**"Unknown"**`
  - Maintains consistency in categorical data
- **Fill missing completion percentage**
  - Assumes missing values indicate `**0% completion**`
  - Prevents issues during analysis and visualization
- **Handle missing Project End Date**
  - Replaces missing values with the `**current date**`
  - Useful for ongoing projects
- **Create Project Duration feature**
  - Calculates project duration in `**days**`
  - Helps analyze timelines, delays, and project performance

In [200…

```
df.head()
```

Out[200…

	Zoning Authority	Project Status	Developer Name	Project Type	Project Name	Project End Date	Escr Age Nar
0	Dubai Development Authority	FINISHED	TECOM Investments Free Zone LLC	buildings	Celia Residence	2024-09-14	Emirat NBD Ba (P.S.
1	Licenses	ACTIVE	Nakheel Company (P.S.C.)	villas	Jebel Ali Townhouses Village - Phase 3	2025-12-31	Mashr Ba (Public Jo Sto Compar
2	Dubai Development Authority	FINISHED	Front Line Investment Management LLC	villas	Damac Hills (2) - Baswood	2022-03-21	Abu Dha Commerc Ba
3	Dubai Development Authority	ACTIVE	Emaar Properties (P.S.C.)	buildings	Benghatti Skyblade	2027-11-30	Dul Islan Ba (Public Jo Sto Compar
4	Licenses	FINISHED	Jumeirah Village (LLC)	buildings	Cappadocia	2013-08-15	First A Dhabi Ba PJ

In [201…

```
df.shape
```

Out[201…

```
(3034, 16)
```

In [202…

```
df.rename(columns={"Project Status.1": "Construction Status"}, inplace=True)
df.head()
```

Out[202…

	Zoning Authority	Project Status	Developer Name	Project Type	Project Name	Project End Date	Escr Age Nar
0	Dubai Development Authority	FINISHED	TECOM Investments Free Zone LLC	buildings	Celia Residence	2024-09-14	Emirat NBD Ba (P.S.
1	Licenses	ACTIVE	Nakheel Company (P.S.C.)	villas	Jebel Ali Townhouses Village - Phase 3	2025-12-31	Mashr Ba (Public Jo Sto Compar
2	Dubai Development Authority	FINISHED	Front Line Investment Management LLC	villas	Damac Hills (2) - Baswood	2022-03-21	Abu Dha Commerc Ba
3	Dubai Development Authority	ACTIVE	Emaar Properties (P.S.C.)	buildings	Benghatti Skyblade	2027-11-30	Dul Islan Ba (Public Jo Sto Compar
4	Licenses	FINISHED	Jumeirah Village (LLC)	buildings	Cappadocia	2013-08-15	First A Dhabi Ba PJ

In [203…

```
# Standardized the text values in categorical columns to maintain consistency.
# to lowercase helps avoid duplicate categories caused by inconsistent capital
df["Project Status"] = df["Project Status"].str.upper()
df["Construction Status"] = df["Construction Status"].str.lower()
df["Project Type"] = df["Project Type"].str.lower()
```

Standardizing Categorical Data

- **Convert Project Status to `uppercase`**
  - Ensures all values are in a consistent format (e.g., "completed", "Completed" → "COMPLETED")
  - Prevents duplicate categories during analysis
- **Convert Construction Status to `lowercase`**
  - Standardizes text values (e.g., "Ongoing", "ONGOING" → "ongoing")
  - Improves grouping and filtering accuracy
- **Convert Project Type to `lowercase`**
  - Maintains uniformity across categories

        - Avoids inconsistencies in visualizations and aggregation

Purpose:
Standardizing text values ensures **data consistency**, reduces errors, and improves the accuracy of **grouping, filtering, and analysis**.

In [204…
```
# Cleaned and standardized the column names by replacing spaces with underscor
#  This improves readability and makes the column names easier to use in Pytho
df.columns = df.columns.str.replace(' ', '_')
df.columns = df.columns.str.replace(':', '')
df.columns = df.columns.str.lower()
```

Cleaning and Standardizing Column Names

- **Replace spaces with underscores**
    - Makes column names easier to use in code
- **Remove special characters (colons)**
    - Cleans unwanted symbols from column names
    - Prevents errors during data manipulation
- **Convert column names to lowercase**
    - Ensures consistency (e.g., *Project_Name* → *project_name*)
    - Improves readability and avoids case-sensitive issues

Purpose:
Standardizing column names improves **code readability, consistency, and ease of access** when performing data analysis and transformations.

In [205…
```
df["escrow_agent_name_"].fillna("Unknown", inplace=True)
```

In [206…
```
# Converted relevant columns such as number of buildings, units, villas, and p
# were converted into missing values (NaN) to ensure accurate calculations and
num_cols = ["noof_buildings","noof_units","noof_villas","percent_completed"]
for col in num_cols:
    df[col] = pd.to_numeric(df[col], errors="coerce")
```

Converting Columns to Numeric Format

- **Define numeric columns**
    - Specifies columns that should contain numerical values
- **Convert columns to numeric type**
    - Converts each column to a numeric data type

- **Handle invalid values**
    - `errors='coerce'` converts non-numeric values into **NaN**
    - Prevents errors during calculations

Purpose:
Ensures data is in the correct format for **analysis, aggregation, and statistical calculations**, improving accuracy and reliability.

In [207…
```
# Handle invalid percent_completed values WITHOUT dropping rows
df.loc[(df["percent_completed"] < 0) | (df["percent_completed"] > 100), "perce

# Fill missing values (use median for better accuracy)
df["percent_completed"].fillna(df["percent_completed"].median(), inplace=True)
```

Filtering Invalid Values

- **Apply upper limit (`≤ 100`) and lower limit (`≥ 0`)**
    - Removes records where completion percentage is greater than 100 and negative completion percentages.
    - Ensures values stay within a logical maximum maintaining logical consistency

Purpose:
Filters out unrealistic values to ensure **data accuracy, reliability, and integrity** for meaningful analysis.

In [208…
```
# Extracted the year from the project_start_date column and created a new colu
#  launches across different years.
df["start_year"] = df["project_start_date"].dt.year
```

Feature Engineering: Extracting Start Year

- **Extract year from project start date**
    - Retrieves the **year** from the project start date column
- **Create a new column**
    - Stores the extracted year in a new column called **start_year**
    - Adds a useful feature for time-based analysis

Purpose:
Helps analyze **year-wise trends** in real estate project launches and identify patterns over time.

In [209…

```
# Examined the unique values in important categorical columns to understand th
print("Unique values of Project Status =", df["project_status"].unique())
print("\nUnique values of Construction Status =",df["construction_status"].uni
print("\nUnique values of Project Type =",df["project_type"].unique())
print("\nUnique values of Zoning authority =",df["zoning_authority"].unique())
```

```
Unique values of Project Status = ['FINISHED' 'ACTIVE' 'NOT_STARTED' 'PENDING'
'CONDITIONAL_ACTIVATING'
 'FREEZED']

Unique values of Construction Status = ['success' 'effective' 'under constructi
on' 'under registration'
 'conditional activation' 'frozen']

Unique values of Project Type = ['buildings' 'villas' 'villa complex']

Unique values of Zoning authority = ['Dubai Development Authority' 'Licenses'
'Dubai Municipality'
 'Dubai Silicon Oasis Authority' 'Dubai South' 'Unknown']
```

Exploring Unique Values in Categorical Columns

- **Check unique values in Project Status**
  - Displays all distinct values in the *project_status* column
  - Helps identify categories like completed, ongoing, etc.
- **Check unique values in Construction Status**
  - Reveals different construction stages
  - Useful for understanding project progress
- **Check unique values in Project Type**
  - Shows different types of projects (e.g., residential, commercial)
  - Helps in categorizing projects
- **Check unique values in Zoning Authority**
  - Identifies different governing authorities
  - Useful for location-based or regulatory analysis

Purpose:
Helps understand **data distribution**, detect **inconsistencies**, and identify **duplicate or incorrect category labels** before analysis.

In [210…

```
# Removed unnecessary leading and trailing spaces from column names and text v
df.columns = df.columns.str.strip()
text_cols = df.select_dtypes(include="object").columns
for col in text_cols:
    df[col] = df[col].str.strip()
```

Removing Extra Spaces from Data

- **Clean column names**
  - Removes leading and trailing spaces from column names
  - Prevents errors when accessing columns
- **Identify text columns**
  - Selects all columns containing text (object type)
  - Prepares them for cleaning
- **Remove spaces from text values**
  - Removes unwanted spaces from the beginning and end of text values
  - Ensures consistency across data entries

Purpose:
Eliminates hidden spaces to improve **data consistency**, ensuring accurate **grouping, filtering, and analysis**.

In [211…

```
# Performed a final validation of the dataset by reviewing data types, checkin
# This step ensures the dataset is properly cleaned and ready for exploratory
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 3034 entries, 0 to 3038
Data columns (total 17 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   zoning_authority         3034 non-null   object
 1   project_status           3034 non-null   object
 2   developer_name           3034 non-null   object
 3   project_type             3034 non-null   object
 4   project_name_            3034 non-null   object
 5   project_end_date         3034 non-null   datetime64[ns]
 6   escrow_agent_name_       3034 non-null   object
 7   noof_buildings           3034 non-null   int64
 8   construction_status      3034 non-null   object
 9   project_start_date       3034 non-null   datetime64[ns]
 10  master_project           3034 non-null   object
 11  noof_units               3034 non-null   int64
 12  noof_villas              3034 non-null   int64
 13  area_name_               3034 non-null   object
 14  percent_completed        3034 non-null   float64
 15  project_duration_(days)  3034 non-null   int64
 16  start_year               3034 non-null   int32
dtypes: datetime64[ns](2), float64(1), int32(1), int64(4), object(9)
memory usage: 414.8+ KB
```

In [212…

```
df.isnull().sum()
```

Out[212…

```
zoning_authority           0
project_status             0
developer_name             0
project_type               0
project_name_              0
project_end_date           0
escrow_agent_name_         0
noof_buildings             0
construction_status        0
project_start_date         0
master_project             0
noof_units                 0
noof_villas                0
area_name_                 0
percent_completed          0
project_duration_(days)    0
start_year                 0
dtype: int64
```

In [213…

```
df.describe()
```

Out[213…

	project_end_date	noof_buildings	project_start_date	noof_units	no
count	3034	3034.000000	3034	3034.000000	303
mean	2022-12-04 00:18:54.622911744	1.374753	2019-08-14 11:09:41.410679040	232.446605	2
min	2000-08-28 00:00:00	0.000000	1998-01-01 00:00:00	0.000000	
25%	2019-08-01 18:00:00	1.000000	2015-04-01 00:00:00	14.250000	
50%	2025-07-05 00:00:00	1.000000	2022-10-03 00:00:00	141.000000	
75%	2027-02-28 18:00:00	1.000000	2024-10-09 06:00:00	300.000000	
max	2031-12-31 00:00:00	199.000000	2026-07-01 00:00:00	10853.000000	178
std	NaN	4.492104	NaN	369.735329	10

In [214…

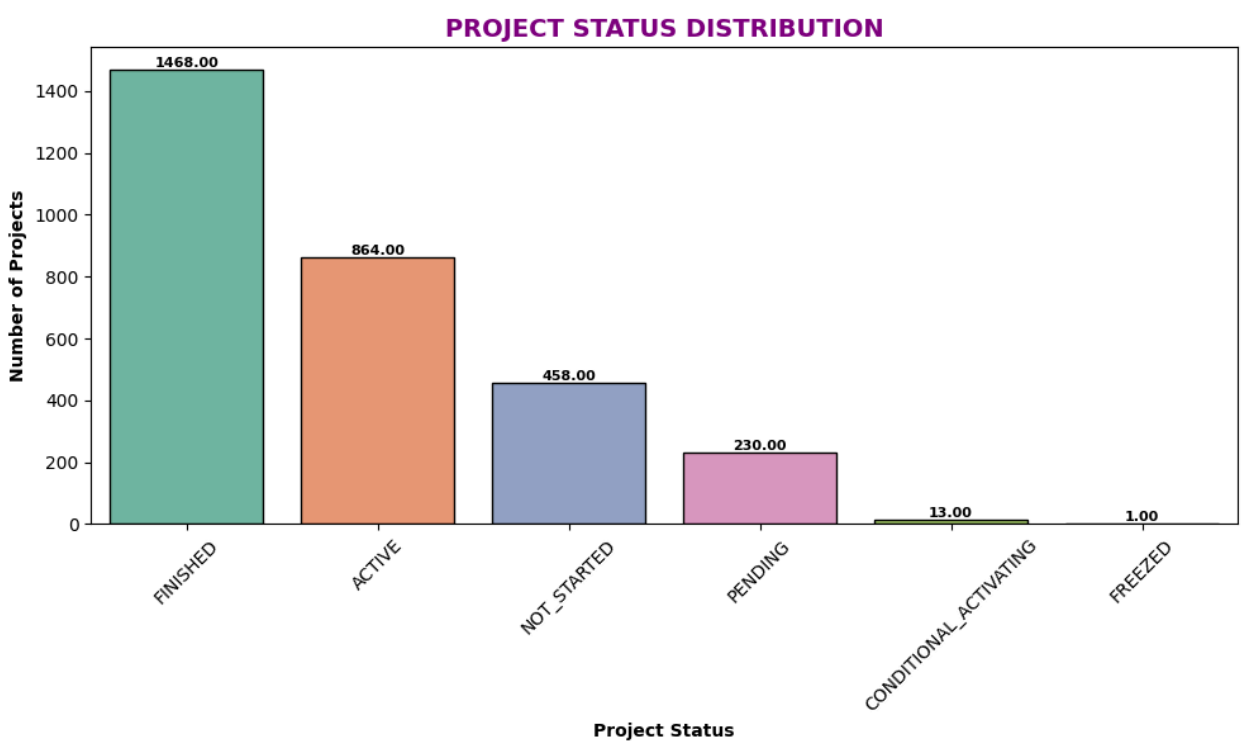
```
df.head()
```

Out[214…

	zoning_authority	project_status	developer_name	project_type	project_name
0	Dubai Development Authority	FINISHED	TECOM Investments Free Zone LLC	buildings	Celia Residenc
1	Licenses	ACTIVE	Nakheel Company (P.S.C.)	villas	Jebel A Townhouse Village - Phas
2	Dubai Development Authority	FINISHED	Front Line Investment Management LLC	villas	Damac Hills (2 - Baswoc
3	Dubai Development Authority	ACTIVE	Emaar Properties (P.S.C.)	buildings	Benghat Skyblac
4	Licenses	FINISHED	Jumeirah Village (LLC)	buildings	Cappadoc

In [215…

```
# Project Status Distribution
plt.figure(figsize=(10,6))
ax=sns.countplot(data=df, x='project_status', hue="project_status",palette='Se
for p in ax.patches:
    ax.annotate(f"{p.get_height():.2f}",(p.get_x() + p.get_width() / 2., p.get
plt.title("PROJECT STATUS DISTRIBUTION",fontweight="bold",color="purple",fonts
plt.ylabel("Number of Projects",fontweight="bold")
plt.xlabel("Project Status",fontweight="bold")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Description:

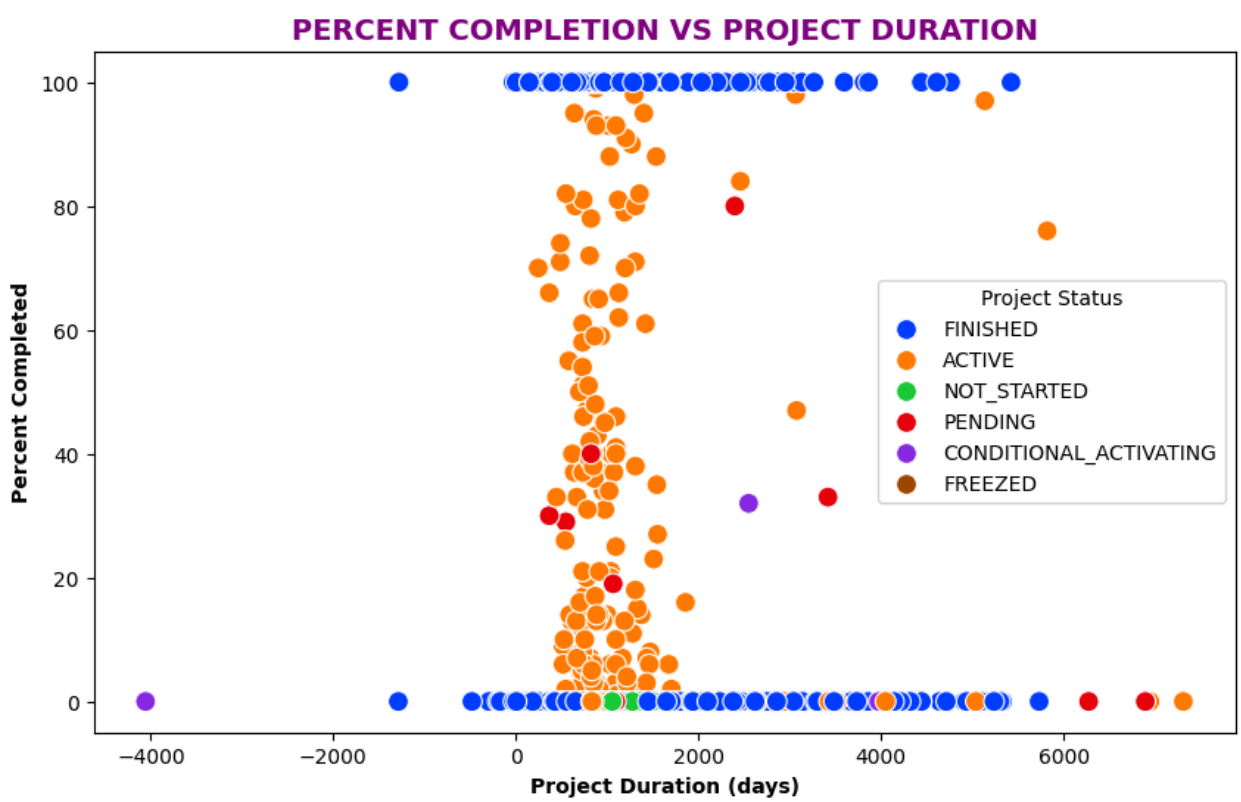
- This bar chart shows the **number of projects in each project status category**.

Key Insights:

- `COMPLETED` projects dominate, showing a significant portion of Dubai's projects are finished.
- `ONGOING` and `FREEZED` projects indicate areas where development is still active or temporarily halted.
- Helps stakeholders understand the `overall progress and identify risk areas`.

In [216…

```
# Percent Completion vs Project Duration (scatter)
plt.figure(figsize=(10,6))
sns.scatterplot(data=df, x='project_duration_(days)', y='percent_completed', h
plt.title("PERCENT COMPLETION VS PROJECT DURATION",fontweight="bold",color="pu
plt.xlabel("Project Duration (days)",fontweight="bold")
plt.ylabel("Percent Completed",fontweight="bold")
plt.legend(title="Project Status")
plt.show()
```



Description:

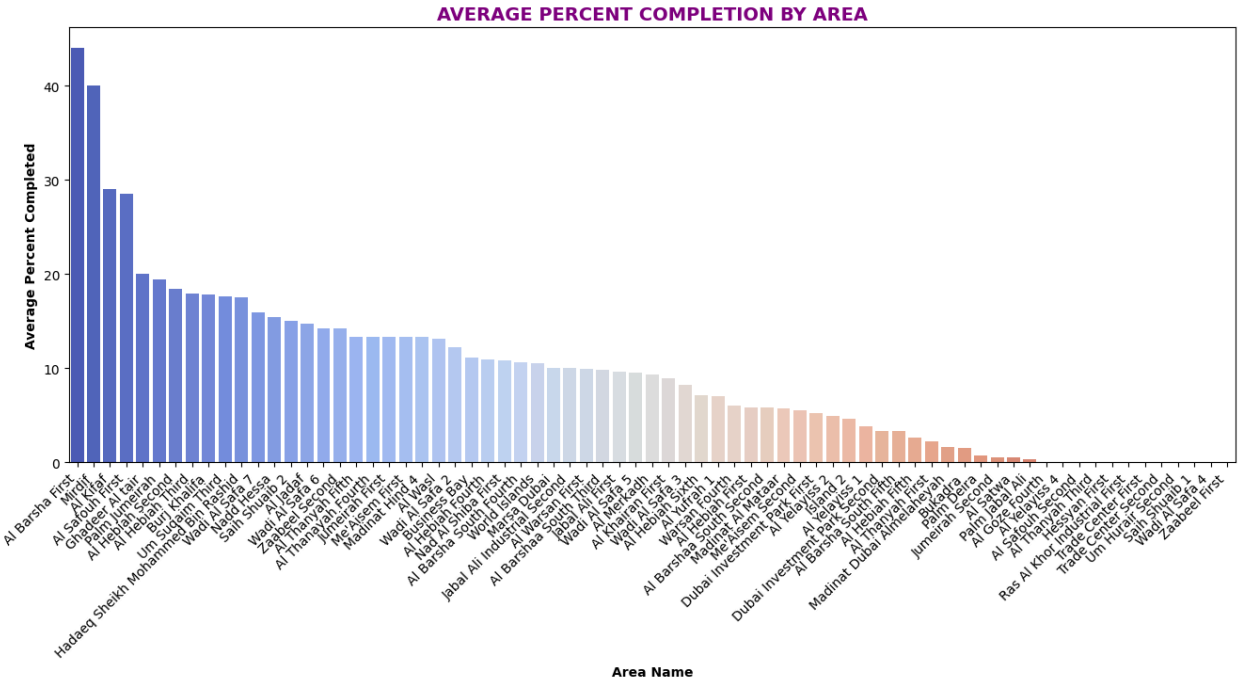
- Scatter plot showing the **relationship between project duration and percent completion**.

Key Insights:

- Projects with **longer durations** do not always have higher completion percentages, indicating `potential delays`.
- Clusters of projects with **low completion and long duration** may require `special attention`.
- Useful for identifying `fast-tracked vs delayed projects`.

In [217…

```
# Average Percent Completed by Area (bar)
avg_completion_area = df.groupby('area_name_')['percent_completed'].mean().sor
plt.figure(figsize=(16,6))
sns.barplot(x=avg_completion_area.index, y=avg_completion_area.values,hue=avg_
plt.title("AVERAGE PERCENT COMPLETION BY AREA",fontweight="bold",color="purple
plt.ylabel('Average Percent Completed',fontweight="bold")
plt.xlabel('Area Name',fontweight="bold")
plt.xticks(rotation=45, ha='right')
plt.show()
```



Description:

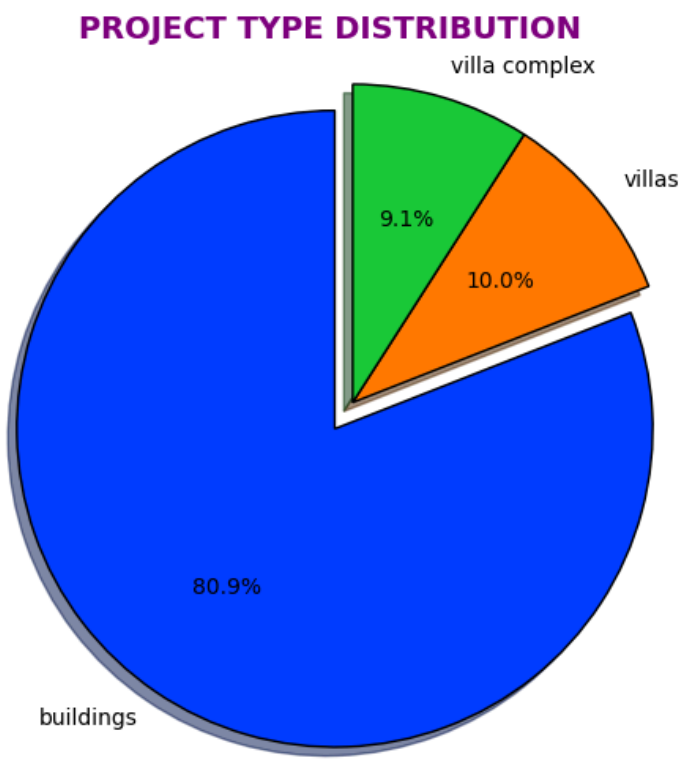
- Bar chart showing the **average percent completion of projects by area in Dubai**.

Key Insights:

- Certain areas have `higher average completion`, indicating `efficient project execution`.
- Areas with `lower average completion` could indicate `development delays or resource constraints`.
- Helps prioritize areas for `investment or policy intervention`.

In [218…

```
#Pie Chart: Project Type Distribution
plt.figure(figsize=(10,6))
project_type_counts = df['project_type'].value_counts()
colors = sns.color_palette('bright')
explode = [0.1 if val == project_type_counts.max() else 0 for val in project_t
plt.pie(project_type_counts,labels=project_type_counts.index,startangle=90,aut
plt.title("PROJECT TYPE DISTRIBUTION ",color="purple",fontweight="bold",fontsi
plt.axis("equal")
plt.show()
```



Description:

- Pie chart showing the **proportion of different project types**.

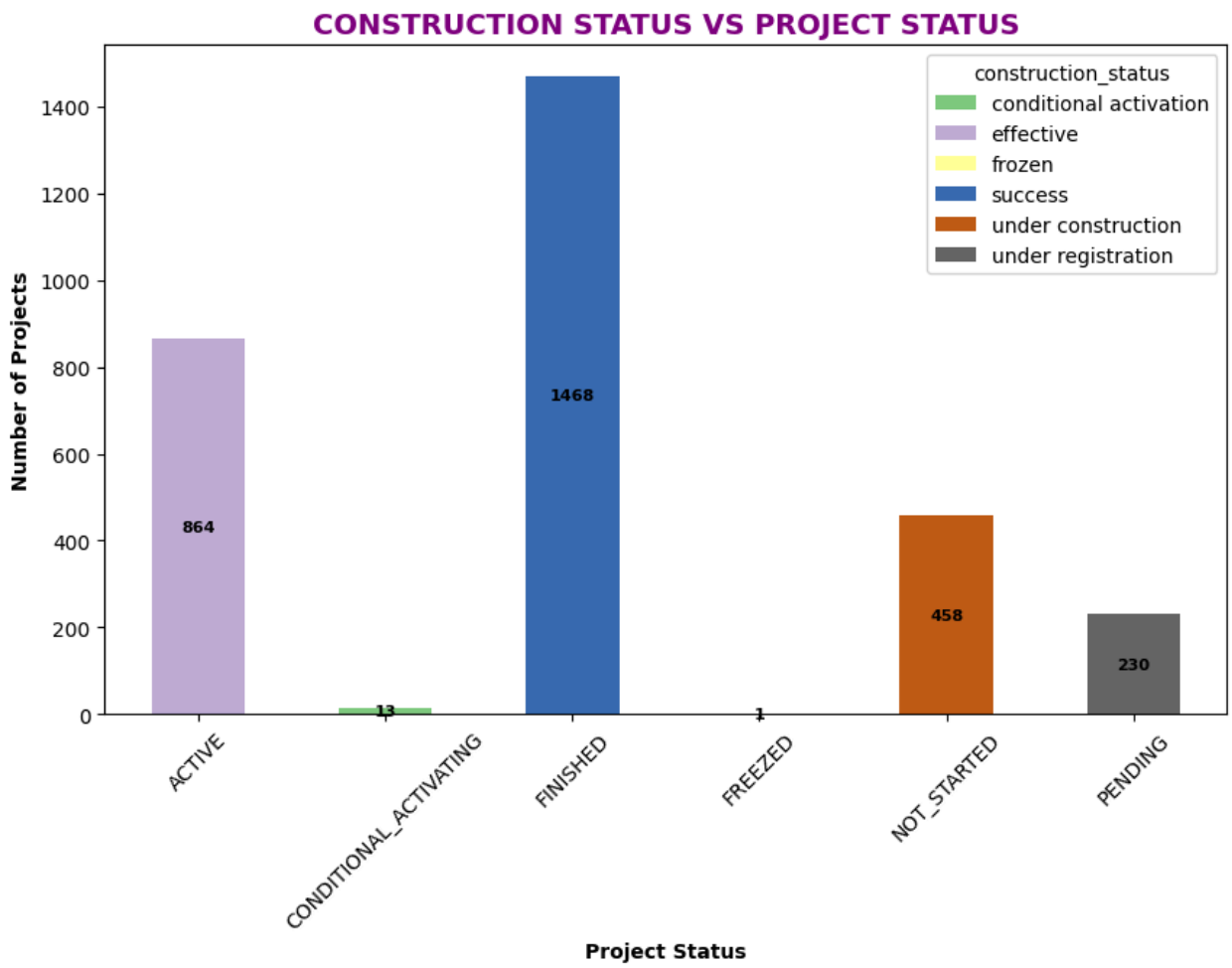
Key Insights:

- `Buildings` are the most common project type, followed by `villas`.
- Reflects the `market focus on multi-unit residential projects`.
- Useful for `investors to understand sector trends and demand`.

In [219…

```
# Construction Status vs Project Status (stacked bar)
plt.figure(figsize=(10,6))
construction_status_counts = df.groupby(['project_status','construction_status
ax = construction_status_counts.plot(kind='bar', stacked=True, figsize=(10,6),
for p in ax.patches:
    if p.get_height() > 0:
        ax.annotate(f"{int(p.get_height())}",(p.get_x() + p.get_width() / 2.,
plt.title("CONSTRUCTION STATUS VS PROJECT STATUS", fontweight="bold", color="p
plt.ylabel('Number of Projects', fontweight="bold")
plt.xlabel('Project Status', fontweight="bold")
plt.xticks(rotation=45)
plt.show()
```

```
<Figure size 1000x600 with 0 Axes>
```



Description:

- Stacked bar chart comparing **construction progress across different project status categories**.

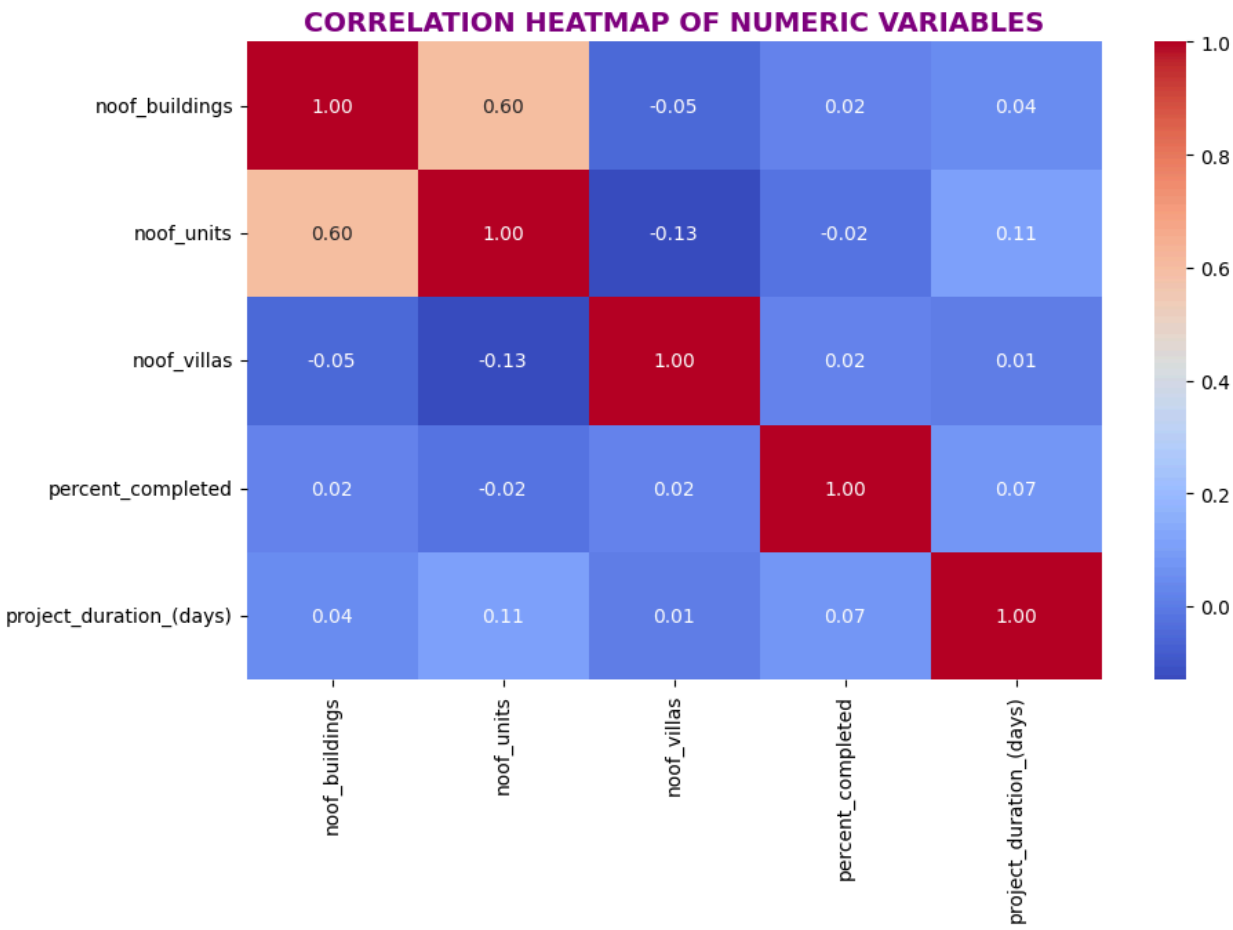
Key Insights:

- Projects labeled as `COMPLETED` mostly have construction marked as `SUCCESS`.

- `ONGOING` projects are spread across various construction stages.
- Highlights `bottlenecks or delays in project execution`.

In [220…

```
# Correlation heatmap for numeric variables
numeric_cols = ['noof_buildings','noof_units','noof_villas','percent_completed
plt.figure(figsize=(10,6))
sns.heatmap(df[numeric_cols].corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title('CORRELATION HEATMAP OF NUMERIC VARIABLES',fontweight="bold", color=
plt.show()
```



Description:

- This heatmap visualizes the **correlation between numeric variables** in the dataset: number of buildings, units, villas, percent completed, and project duration.

Key Insights:

- `noof_buildings` and `noof_units` have a **moderate positive correlation (~0.60)**, indicating larger projects generally have more units.

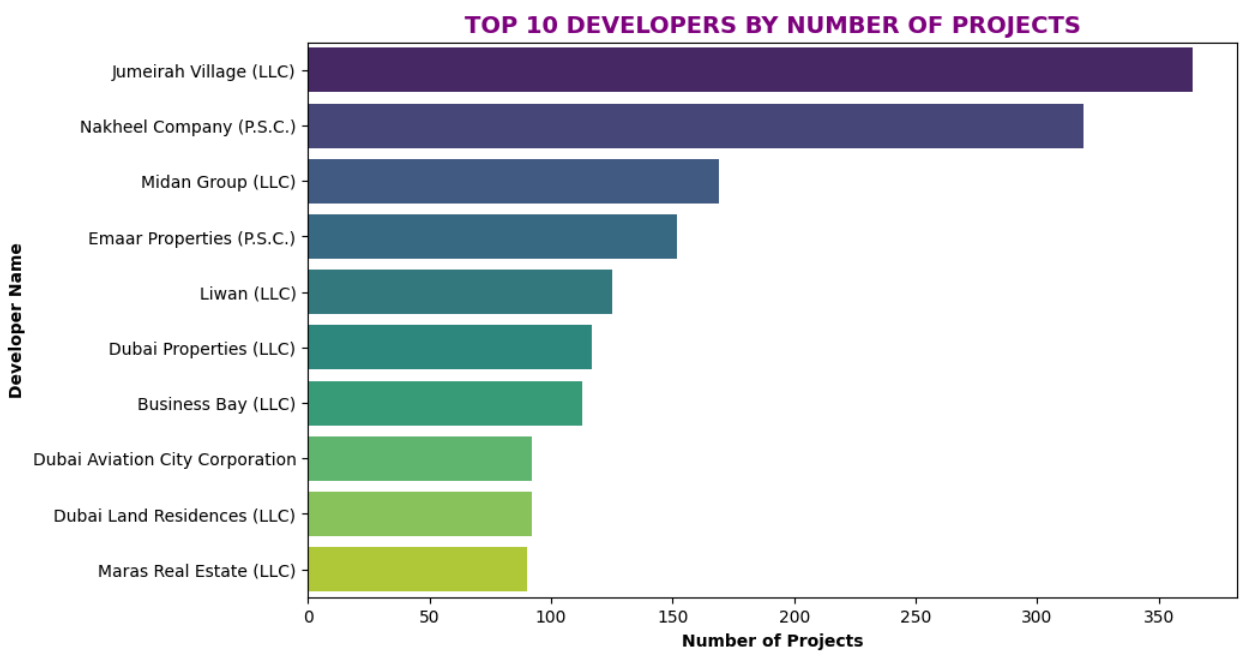
- `noof_villas` has **near-zero or slightly negative correlations** with other numeric variables, suggesting villas are handled differently from larger building/unit projects.
- `percent_completed` shows **very low correlation** with all other numeric variables, meaning project size or duration does **not directly determine completion percentage**.
- `project_duration_(days)` shows a **weak positive correlation (~0.11) with number of units**, indicating larger projects might take slightly longer.

Purpose:

- Helps identify relationships between key numeric features, which is useful for `predictive modeling`, `resource allocation`, and `prioritizing monitoring of project completion`.

In [221…

```
# Top Developers by Number of Projects
top_developers = df['developer_name'].value_counts().head(10)
plt.figure(figsize=(10,6))
sns.barplot(x=top_developers.values, y=top_developers.index, palette='viridis'
plt.title('TOP 10 DEVELOPERS BY NUMBER OF PROJECTS',fontweight="bold", color="
plt.xlabel('Number of Projects',fontweight="bold")
plt.ylabel('Developer Name',fontweight="bold")
plt.show()
```



Description:

- Horizontal bar chart showing the **top 10 developers by number of**

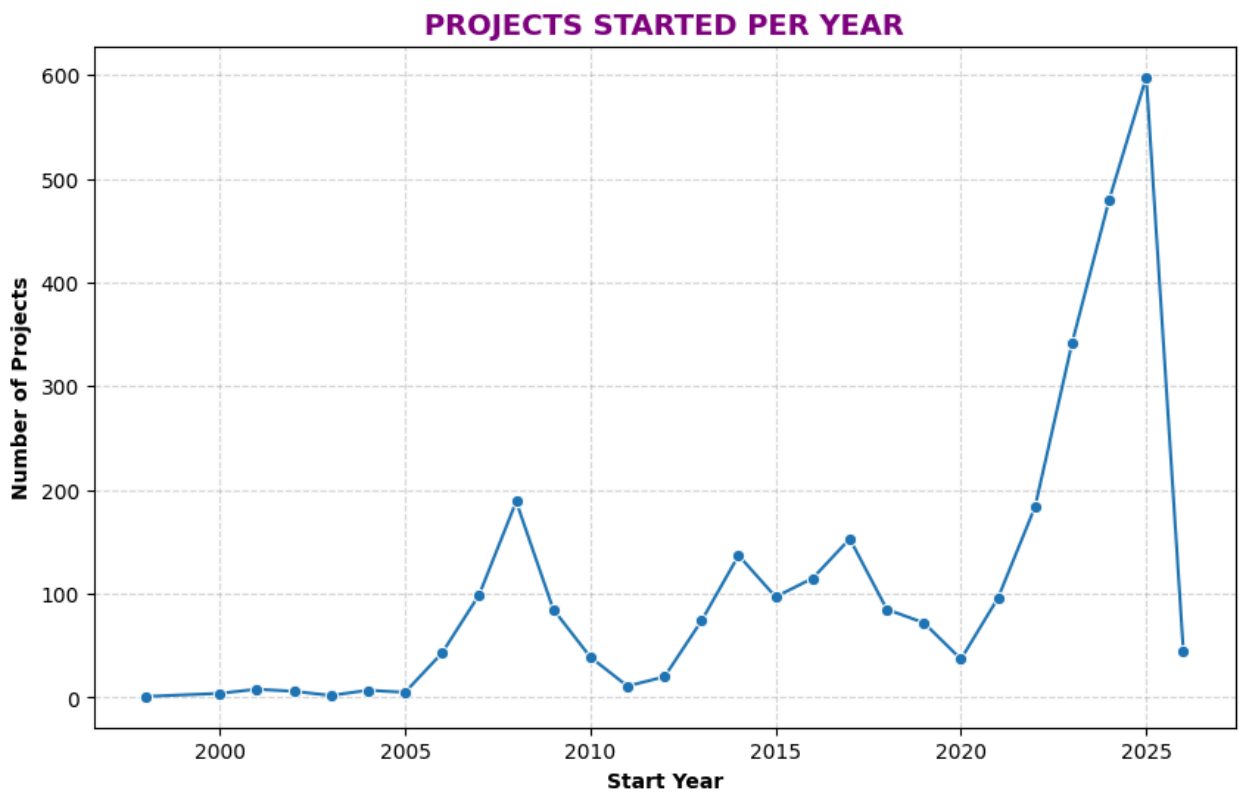
projects.

Key Insights:

- Certain developers dominate the market, reflecting `higher influence and market share`.
- Helps identify `key players for partnerships or competition analysis`.
- Highlights `industry concentration` in Dubai's real estate sector.

In [222…

```
# Explore development trends over years
projects_per_year = df['start_year'].value_counts().sort_index()
plt.figure(figsize=(10,6))
sns.lineplot(x=projects_per_year.index, y=projects_per_year.values, marker='o'
plt.title('PROJECTS STARTED PER YEAR',fontweight="bold", color="purple", fonts
plt.xlabel('Start Year',fontweight="bold")
plt.ylabel('Number of Projects',fontweight="bold")
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```



Description:

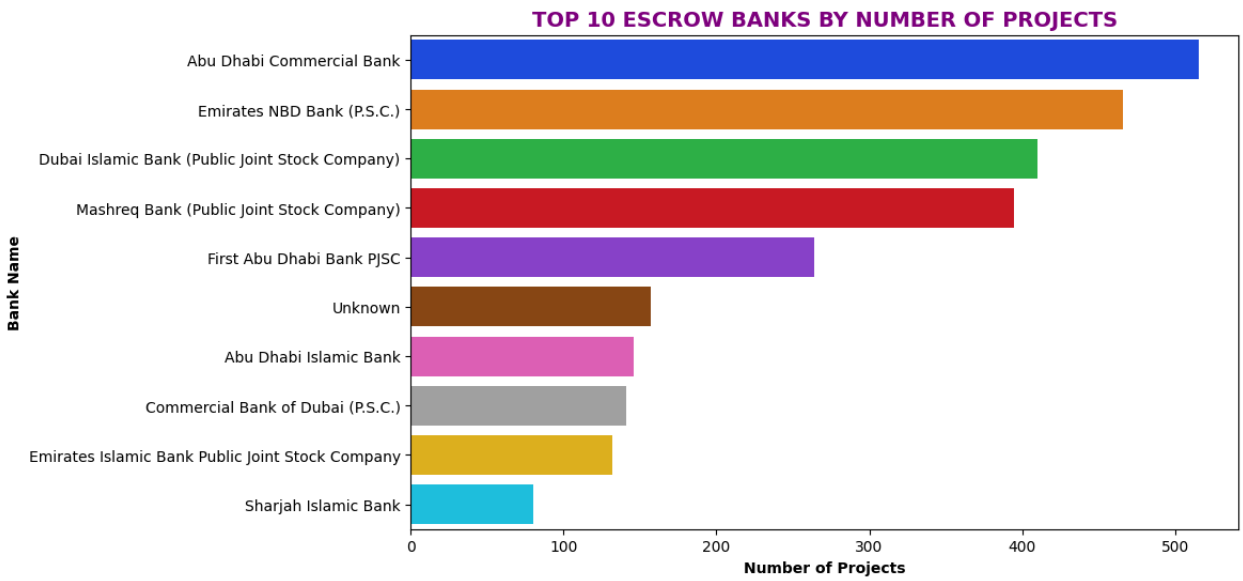
- Line chart showing the **number of projects started per year**.

Key Insights:

- Peaks in certain years indicate `periods of rapid real estate development`.
- Declines may correspond to `market slowdowns or economic factors`.
- Useful for `analyzing trends and predicting future project pipelines`.

In [223…

```
# Top Banks acting as Escrow Agents
top_banks = df['escrow_agent_name_'].value_counts().head(10)
plt.figure(figsize=(10,6))
sns.barplot(x=top_banks.values, y=top_banks.index, palette='bright',hue=top_ba
plt.title('TOP 10 ESCROW BANKS BY NUMBER OF PROJECTS',fontweight="bold", color
plt.xlabel('Number of Projects',fontweight="bold")
plt.ylabel('Bank Name',fontweight="bold")
plt.show()
```



Description:

- Bar chart showing the **top banks acting as escrow agents**.

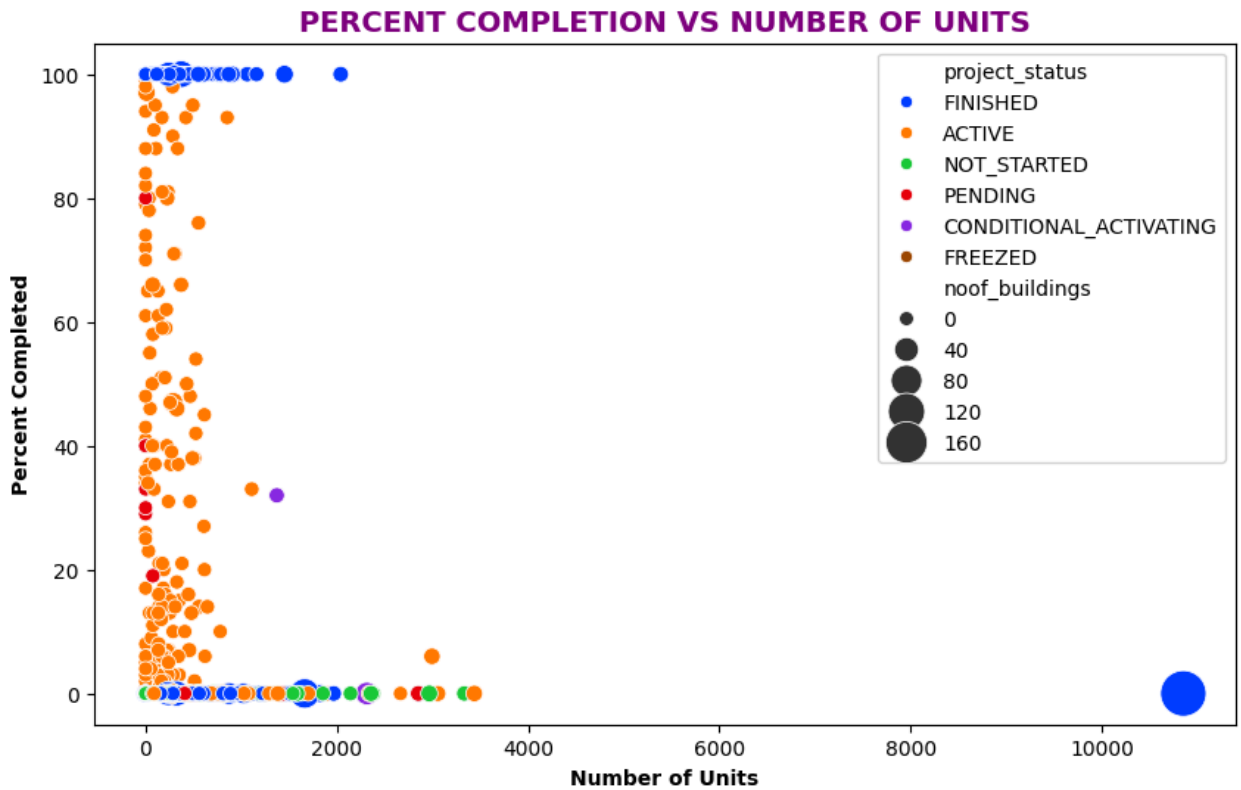
Key Insights:

- Identifies `banks most trusted by developers for financial management`.
- Highlights `financial infrastructure supporting real estate projects`.
- Useful for `developers, investors, and policymakers` to understand banking trends.

In [224…

```
# Scatterplot with size & hue
```

```
plt.figure(figsize=(10,6))
sns.scatterplot(data=df, x='noof_units', y='percent_completed', hue='project_s
plt.title("PERCENT COMPLETION VS NUMBER OF UNITS",fontweight="bold", color="pu
plt.xlabel("Number of Units",fontweight="bold")
plt.ylabel("Percent Completed",fontweight="bold")
plt.show()
```



Description:

- Scatter plot showing the **relationship between project size (number of units) and percent completion**.

Key Insights:

- Larger projects tend to take longer to reach full completion, showing `scale-related delays`.
- Clusters of `small projects with high completion` indicate faster execution for smaller developments.
- Helps assess `construction efficiency relative to project size`.

In [225…

```
# Create subplots
fig = make_subplots(rows=2, cols=2,subplot_titles=("<b>Project Duration Distri
                                                   "<b>Project Duration by Typ

# 1⃣ Histogram - Project Duration
```

```
fig.add_trace(go.Histogram(x=df['project_duration_(days)'],nbinsx=30,marker=di

# 2⃣ Bar Plot- completion categories
bins = [0, 25, 50, 75, 100]
labels = ['0-25%', '25-50%', '50-75%', '75-100%']
df['completion_category'] = pd.cut(df['percent_completed'], bins=bins, labels=
completion_counts = df['completion_category'].value_counts().sort_index()
fig.add_trace(go.Bar(x=completion_counts.index,y=completion_counts.values,mark

# 3⃣ Line Plot - Projects per Year
yearly = df['start_year'].value_counts().sort_index()
fig.add_trace(go.Scatter(x=yearly.index,y=yearly.values,mode='lines+markers',l

# 4⃣ Box Plot - Duration by Project Type
for ptype in df['project_type'].dropna().unique():
    fig.add_trace(go.Box(y=df[df['project_type'] == ptype]['project_duration_(

# Axis labels
fig.update_xaxes(title_text="Project Duration (Days)", row=1, col=1)
fig.update_yaxes(title_text="Number of Projects", row=1, col=1)

fig.update_xaxes(title_text="Completion Category", row=1, col=2)
fig.update_yaxes(title_text="Number of Projects", row=1, col=2)

fig.update_xaxes(title_text="Year", row=2, col=1)
fig.update_yaxes(title_text="Number of Projects", row=2, col=1)

fig.update_xaxes(title_text="Project Type", row=2, col=2)
fig.update_yaxes(title_text="Project Duration (Days)", row=2, col=2)

# Layout
fig.update_layout(height=800,width=1000,title_text="<b>📊 Dubai Real Estate Ir
fig.show()
```

1. Project Duration Distribution

- Most projects fall within a specific duration range, indicating standardized timelines.
- Presence of long-duration projects suggests delays or large-scale developments.

2. Completion Category Analysis

- A significant number of projects fall in the lower completion ranges (0–50%), indicating many projects are still in early stages.
- Fewer projects in the 75–100% range highlight that full completion takes considerable time.
- Helps stakeholders identify project pipeline stages and progress distribution.

3. Projects Started Over Years

- Project launches show fluctuations over time, indicating changing market conditions.
- Peaks represent periods of rapid development, while declines may reflect economic slowdowns.

4. Project Duration by Type (Box Plot)

- Different project types exhibit variation in duration and consistency.
- Some project types show wide spread and outliers, indicating unpredictable timelines.
- Helps identify which project types are more stable vs high-risk in execution.

🎤 📊 Analytical Insights Overview

📊 1. Majority of projects are COMPLETED, indicating strong execution and market maturity

⏳ 2. Longer duration ≠ higher completion → highlights delays and inefficiencies

📍 3. Some areas show high efficiency, while others indicate development gaps

🏢 4. Buildings dominate, reflecting high-density urban development focus

🏗️ 5. Completed projects align with successful construction stages and Ongoing projects show mixed progress levels

📊 6. Weak correlation between completion and other variables → external factors matter more

🏢 7. Few developers dominate → high market concentration

📈 8. Fluctuations indicate cyclical market trends and economic influence

🏦 9. Limited banks handle most projects → strong financial dependency

📊 10. Larger projects tend to have slower completion, showing scale challenges

🎯 Final Conclusion

- Dubai’s real estate sector is strong, dynamic, and expanding
- However, project delays and uneven development still exist
- Improving efficiency, planning, and monitoring is critical

👉 Data-driven insights can help investors, developers, and policymakers make smarter decisions and drive sustainable growth